

Technical Report: Book Library and Analytics API

1. Introduction

This project is a RESTful HTTP backend centered around a book catalog. Bibliographic information is stored in a relational database, and clients primarily interact with the API via JSON. For book data, the interface supports four common operations: adding a new book record, querying by condition or ID, updating an existing record, and deleting a record. These four operations are collectively referred to as CRUD in the following text.

In addition to the above routine maintenance capabilities, the service also provides a statistics-type interface. The interface groups records by the genre field on each book, and aggregates for each genre the total number of books, the average rating, and so on. This interface only allows querying the results; statistics cannot be directly modified through the interface. The numbers are calculated in real time by the backend based on the current data in the table. There is also a simple endpoint that quickly answers questions such as whether the service has started and can respond to requests. This endpoint returns minimal content and generally does not access business tables, making it convenient for deployment and liveness probe scripts. The project's root path provides a brief web page that conveniently opens Swagger in a browser. The project also provides OpenAPI-related documentation paths.

2. Architecture and Key Design Choices

The overall approach follows the classic three-tier architecture. The client accesses FastAPI routes via HTTP; the application layer obtains database sessions through dependency injection and calls CRUD operations. The persistence layer is handled by the SQLAlchemy Session and an engine pointed to by SQLite or any other DATABASE_URL. Application-layer routes are split into two API Routers: books and analytics, which are mounted under /api/v1 in main.py, making it easy to evolve the API without breaking existing clients.

In terms of module division, main.py handles the application lifecycle and global exception handling. config.py centralises Settings. database.py provides the engine, session factory, and get_db(). models.py defines database table mappings. schemas.py defines JSON validation and structure for API requests and responses. crud.py remains FastAPI-agnostic, making it easy to test separately. deps.py encapsulates authentication and database dependencies as reusable Annotated aliases. The /app/routers directory only orchestrates HTTP semantics. This decoupling reduces coupling and improves maintainability.

For request paths, read operations obtain a session via get_db(), query the ORM, and serialise the result to BookRead. Statistical analytical endpoints are also read-only, require no API key, and the aggregation logic is implemented in analytics_summary() inside the CRUD module. Write operations first execute verify_api_key() when authentication is enabled, then validate the request body with BookCreate or BookUpdate, and finally commit the transaction. A unified RequestValidationError handler is registered, whenever a request parameter or JSON fails validation, it returns a JSON with a code field, distinguishing it from business-oriented 404 errors.

Regarding API style, the project focuses mainly on the single resource, books, with read-only statistics. There is no need for multi-entity graph traversal or client-driven field trimming. Hence REST with OpenAPI first is preferred over introducing GraphQL.

3. Technology Stack Rationale: Language, Framework, and Database

3.1 Programming language: Python 3

The choice of Python is based on the following reasons. First, this project requires rapid iteration and readable source code; Python's syntax and ecosystem are sufficient for such a small service. Second, FastAPI, SQLAlchemy, Pydantic, and pytest integrate smoothly within the same language stack, reducing cross-language boundaries. Third, using Python's type annotations combined with typing_extensions.Annotated to annotate dependencies and parameters makes the routing and injection relationships clearly visible in the IDE. This preserves Python's strengths of rapid development while improving readability and future maintainability through type information.

3.2 Application framework and contracts: FastAPI, Pydantic, Uvicorn

One of the key reasons for choosing FastAPI is that, while implementing REST APIs, the framework generates OpenAPI based on the type annotations of routes and dependencies, and exposes pages such as `/docs` and `/redoc`, thereby reducing the burden of maintaining documentation separately and lowering the risk of inconsistency between documentation and implementation. Pydantic v2 models request and response bodies and settings, aligning BookCreate with column lengths to block invalid writes at the boundary. pydantic-settings loads DATABASE_URL, API_KEY, and DISABLE_AUTH from `.env`, separating config from code.

For local development, start the application with Uvicorn using the common command:

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

In this way, the code will automatically reload after saving, else hot reloading is not needed, remove the `--reload` flag from the command.

Only the most basic uvicorn is written in the `requirements.txt` dependency file. This is because optional Uvicorn libraries often lack pre-built installation packages on Windows, pip will attempt to compile from source locally, which may fail if the machine does not have a compilation environment.

3.3 Persistence: SQLAlchemy 2.x with SQLite as default

The choice of SQLAlchemy 2.x as the Object-Relational Mapping is based on the following. Declarative models describe table structures, and `select()` is used for queries. The CRUD operations for books and the queries behind the analytical endpoints are completed within the same database session. Grouping, counting, averaging, are executed inside the database rather than loading the entire table into Python for statistics.

At the database level, SQLite is used, with the database file `books.db` in the project, and no separate database process. The connection string is externalised via `DATABASE_URL`. When switching to PostgreSQL or other databases, most business SQL can be handled by SQLAlchemy. This design prioritises lowering operational and deployment overhead, accepting the known limitations of SQLite in concurrent writes and lock granularity. It is sufficient when reads are frequent, writes are infrequent, and concurrency is controllable. If write concurrency or data volume increases significantly, migration to a server-class database with connection pooling and formal migration tools should be considered. Currently, the system uses `create_all()` to create tables and does not introduce Alembic. However, if the project enters long-term production and the table structure changes frequently, Alembic should be introduced for formal migrations.

Tests use pytest and Starlette TestClient with in-memory SQLite and `dependency_overrides` so regressions do not touch the developer's `books.db`. httpx is pulled in as a test dependency.

4. Data Model and API Semantics

4.1 Table structure and columns

The database table is named `books`, and the ORM mapping class is `app.models.Book`. The table structure is defined by the SQLAlchemy declarative model, while API requests and responses are validated by the Pydantic schemas `BookCreate`, `BookUpdate`, and `BookRead`.

String columns have minimum and maximum length constraints on the API side, consistent with the VARCHAR width in the database. Columns that allow nulls are NULL in the database, corresponding to the field being omitted or explicitly set to null in JSON. The `rating` field is optional; when calculating the average rating in analytical endpoints, the SQL AVG function is used, and rows with NULL are excluded from the average.

Attribute definitions and relevant information for table `books`:

Attribute	SQL type	Constraints / indexes	API and validation notes
<code>id</code>	INTEGER	Primary key;	❖ Only on BookRead; generated by DB;

		autoincrement,indexed	❖ not in POST body
title	VARCHAR(500)	NOT NULL, indexed	❖ Required; ❖ Range: 1 to 500;
author	VARCHAR(300)	NOT NULL, indexed	❖ Required; ❖ Range: 1 to 300
genre	VARCHAR(120)	NOT NULL, indexed	❖ Required; ❖ Range: 1 to 120;
publication_year	INTEGER	Nullable	❖ Optional; ❖ Range :1000 to 2100;
rating	FLOAT	Nullable	❖ Optional; ❖ Range: 0 to 5 inclusive;
notes	TEXT	Nullable	❖ Optional; ❖ No max length limit

The primary key `id` is used in the REST path `/api/v1/books/{id}`, and the list is sorted by `id` by default. Indexes are built on the three columns `title`, `author`, and `genre` to support filtering by genre and high-frequency list reads. Author and genre are stored as strings together with the same book record, rather than being split into separate author and genre tables, in order to reduce the implementation and data preparation costs for CRUD operations and CSV batch imports. The analytical endpoints perform exact matching and grouping by `genre`. If the same genre appears in different expressions, they are counted as separate groups – a deliberate trade-off to keep the model simple. Rules such as rating ranges and publication year ranges are validated by `Pydantic` at the API layer. If the database is modified directly, bypassing the API, data that does not conform to the business assumptions may be written.

4.2 REST choice and semantics

This project primarily implements two functions: First, providing CRUD operations around the single resource of books. Second, providing read-only statistics aggregated by genre. There are no complex relationships between tables, nor is there a need to combine queries across multiple resources arbitrarily within a single request. Therefore, the common REST API approach is adopted, and `FastAPI` automatically generates `OpenAPI`, making it convenient to test and verify the contract in `Swagger`. `GraphQL` is not used, avoiding the extra maintenance of a query parsing layer and type definitions for the current small data scale.

In terms of common HTTP semantics: repeated identical GET or DELETE requests do not change data outside the target resource. Deleting a non-existent book `id` returns 404 and does not affect other rows. When creating a new book, each successful submission adds one row to the database. Partial updates use PATCH, meaning that only the fields present in the request body are updated; fields not mentioned retain their original values. Therefore, this design is suitable for making small changes to a few fields.

4.3 Pagination, filtering, and status codes

GET `/api/v1/books/` supports `skip`, `limit`, and optional exact `genre` match; `limit` is capped at 100. CRUD orders by `id` for stable ordering. Offset pagination is used and deep pages or heavy concurrent writes may motivate cursor-based pagination. HTTP status codes include 200, 201, 204, 401, 404, and 422. Validation failures include `validation_error` to distinguish them from 404.

5. Security, Configuration, and Data Ingestion

Mutating endpoints use a shared-secret X-API-Key checked against `API_KEY`; `DISABLE_AUTH` can disable this in tests. The model fits demos and single-team use, not multi-tenant revocation. Secrets live in `.env`; the repo ships `.env.example`. Public deployments should terminate HTTPS at the platform.

CSV import is run by `scripts/import_csv.py`. The default dataset is `data/books_gutenberg_catalog_sample.csv`, built by `scripts/build_gutenberg_csv.py` by filtering Project Gutenberg's official catalogue CSV. The script shares `BookCreate` and `create_book()` with HTTP layer so bulk loading follows the same validation rules as the API. UTF-8-sig and `--replace` are supported.

6. Deployment and Operations

Local start command:

```
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Production should use a process manager. Hosting requires ASGI entry, environment variables, and a persistent `DATABASE_URL`. Structured logging and metrics are not implemented; troubleshooting relies on console output and dev tracebacks; production could add request IDs and health checks.

7. Testing, Challenges, and Lessons Learned

7.1 Test approach and coverage

`test_api.py` covers health, full CRUD, analytics JSON shape, validation envelope, length and numeric bounds, genre filtering, and limit overflow. `test_auth.py` uses a separate fixture with auth enabled to assert 401 without a key and 201 with the correct key. `conftest` uses in-memory SQLite, `StaticPool`, overridden `get_db()`, and clears `dependency_overrides` and `get_settings` cache to avoid polluting local `books.db`.

7.2 Challenges during development

First, `pydantic-settings` uses caching to avoid repeatedly reading configuration files. If only environment variables are modified without calling `cache_clear()` in tests, the old `Settings` object will still be returned. Therefore, explicitly clear this cache during teardown in `conftest`.

Second, the design of `Uvicorn` is detailed in Section 3.2.

Third, boundary and query testing, as well as `Swagger` joint debugging, are covered in Sections 7.1 and 4.2.

7.3 Lessons learned

First, database read and write operations are centralized in `CRUD` module, while the routing layer only handles parameters and HTTP semantics. This ensures that integration tests follow the same path as real requests. If more detailed unit tests are added later, the same set of rules in `CRUD` module can be reused. The CSV import script calls the same `create_book()` function as the API, avoiding inconsistencies between import and interface logic. Second, `OpenAPI` serves as the standard principle for the design and implementation of the entire project. Third, tests leverage `FastAPI`'s dependency override mechanism, allowing the use of an in-memory database and isolated environment without major architectural changes. This approach provides the best balance of cost and benefit for the current project scale.

8. Limitations and Potential Future Improvements

Authorisation remains a global `API key`. `OAuth2` or per-client keys stored and revocable in a database are future options. Under write concurrency and growth, migrate to `PostgreSQL` with pooling, backups, and `Alembic`-style migrations. Analytics could move from live aggregates to materialised views or caches. Pagination can evolve from `offset` to cursors. Search can extend beyond exact genre to full-text or dedicated search engines. Richer domains may warrant `GraphQL` or `MCP`-style tooling. Observability should move from console logs to structured logging and alerting.

9. Generative AI: Declaration and Reflective Analysis

9.1 Compliance declaration and tool log

Tool	Primary use	Code	Documentation	Debugging
Cursor (Composer, IDE-embedded assistant)	Implementing and refining the project: feature development, refactoring, tests and runtime troubleshooting, dependency- and environment-related edits	Yes	Partial	Yes
DeepSeek	Chinese–English translation and polishing technical report wording	No	Yes	No

9.2 Thoughtful use of generative tools

Cursor is mainly used for modifying backend code and testing, as well as troubleshooting runtime and dependency issues. DeepSeek is primarily used for writing, refining, and translating the project report. I am still responsible for running pytest locally, comparing the OpenAPI specification with the actual behaviour of the repository, and performing analysis and judgement.

Currently, GenAI is suitable for accelerating implementation and revising drafts, but it may introduce some subtle issues or implementations that deviate from the project design. Therefore, the design document should be used as the standard — changes must be analysed and adopted step by step, and everything should be verified personally, rather than accepting all of AI’s output.

9.3 Chat record screenshot



Figure 1. Cursor chat



Figure 2. Cursor chat



Figure 3. DeepSeek chat

References

1. FastAPI Documentation. <https://fastapi.tiangolo.com/>
2. Pydantic Documentation. <https://docs.pydantic.dev/>
3. SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>
4. Open Library. <https://openlibrary.org/>
5. DeepSeek. <https://chat.deepseek.com>

Deliverables

Github Repository: [youzixiaobai/WebServiceWebData: coursework data](https://github.com/youzixiaobai/WebServiceWebData)

Code: [WebServiceWebData/WebServiceWebData 1.zip at main • youzixiaobai/WebServiceWebData](#)

API document : [WebServiceWebData/API Report.pdf at main • youzixiaobai/WebServiceWebData](#)

PPT : [WebServiceWebData/Technical Report Book Library and Analytics API.pptx at main • youzixiaobai/WebServiceWebData](#)